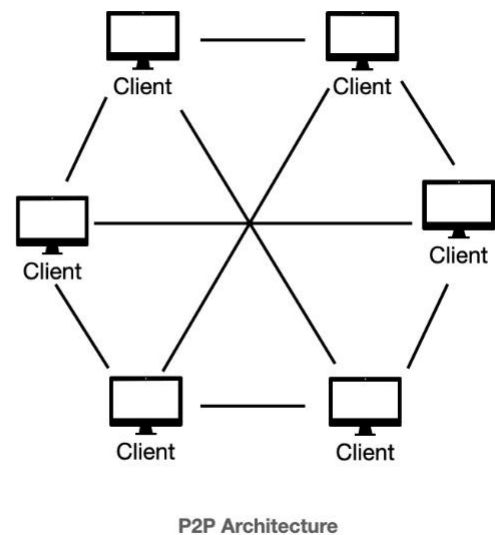
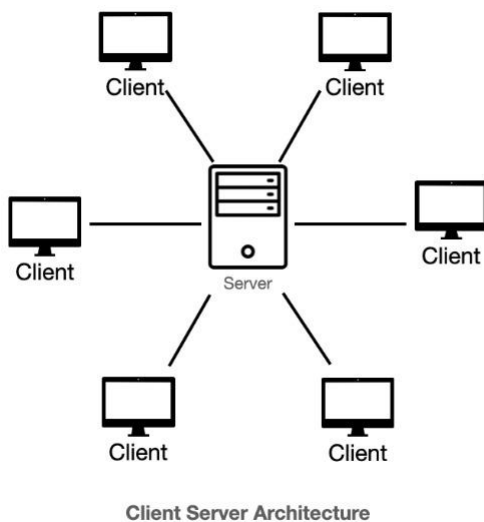


Architecture Pair-a-Pair (P2P)

1. Qu'est-ce que le Pair-a-Pair (P2P) ?

Dans une architecture P2P, il n'y a pas de serveur central. Le réseau est un ensemble de "pairs" (peers) qui sont égaux. Chaque pair agit simultanément comme un client (en demandant des informations) et comme un serveur (en fournissant des informations).

- Les pairs communiquent directement les uns avec les autres.
- Le réseau est décentralisé et distribué.
- Chaque pair apporte ses propres ressources (connexion, puissance de calcul) au réseau.



2. les avantages et les inconvénients :

Type	Critère	P2P	Client-Serveur
Avantages	Résilience	Décentralisé → le réseau continue même si certains peers tombent en panne	Dépend du serveur central → panne du serveur = panne du réseau
	Évolutivité	Chaque nouveau peer apporte des ressources supplémentaires	Le serveur peut être rapidement saturé
	Réduction des coûts	Ressources et maintenance partagées entre les peers	Les coûts sont concentrés sur le serveur central
Inconvénients	Sécurité	Données exposées aux malwares et attaques, protection dépend des peers	Protection centralisée plus facile à gérer
	Légalité	Partage direct de fichiers → risque de violation du droit d'auteur	Le serveur peut contrôler le contenu et limiter les risques légaux
	Gestion des ressources	Dépend des capacités des peers → performances inégales	Serveur central avec ressources dédiées → performance stable

3. Exemple Concret :

Fichier 1 : **Peer.java** (Le Cœur du Pair)

C'est la classe principale qui représente un utilisateur. Elle a deux responsabilités qui illustrent parfaitement le P2P :

- Un rôle de Serveur : Au démarrage, Peer.java lance un ServerSocket. Son unique but est d'écouter et d'accepter les connexions entrantes d'autres pairs.

```

public void startServer() {
    new Thread(() -> {
        try (ServerSocket serverSocket = new ServerSocket(port)) {
            System.out.println("Server started on port " + port + ". Waiting for peers ...");
            while (true) {
                // Accept a new connection
                Socket clientSocket = serverSocket.accept();
                System.out.println("New peer connected from " + clientSocket.getRemoteSocketAddress());

                // Create a new thread to handle this peer's communication
                PeerConnection connection = new PeerConnection(clientSocket, this);
                connections.add(connection);
                connection.start();
            }
        } catch (IOException e) {
            System.err.println("Server error: " + e.getMessage());
        }
    }).start();
}

```

- Un rôle de Client : En parallèle, il permet à l'utilisateur d'initier une connexion sortante vers un autre pair.

```

public void connectToPeer(String host, int port) {
    try {
        Socket socket = new Socket(host, port);
        System.out.println("Successfully connected to " + host + ":" + port);

        // Create a new thread to handle this new connection
        PeerConnection connection = new PeerConnection(socket, this);
        connections.add(connection);
        connection.start();
    } catch (UnknownHostException e) {
        System.err.println("Unknown host: " + host);
    } catch (IOException e) {
        System.err.println("Could not connect to " + host + ":" + port + " (" + e.getMessage() + ")");
    }
}

```

Fichier 2 : **PeerConnection.java** (Le Gestionnaire de Connexion)

Une nouvelle instance de PeerConnection est créée pour chaque connexion individuelle (entrante ou sortante).

Son unique responsabilité est de s'exécuter dans son propre thread pour gérer la communication (envoyer et recevoir des messages) avec un seul autre pair.

```

public void run(){
    try{
        // Initialize input and output streams for this connection
        in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
        out = new PrintWriter(socket.getOutputStream(), true);

        String message;
        // Listen for messages from this peer
        while ((message = in.readLine()) != null){
            // Print the received message to our console
            System.out.println(message);
        }
    } catch (IOException e) { ...
    } finally {
        // Clean up the connection
        closeConnection();
    }
}

```